

Day 6 - CSS Grid

Day 6 — CSS Grid

Objectives

- Build the core mental model: a grid container defining rows AND columns simultaneously, unlike Flexbox's single axis
- Use `grid-template-columns/grid-template-rows` and the `fr` unit
- Place items precisely using named grid lines and `grid-area`
- Know, clearly and confidently, when to reach for Grid instead of Flexbox

Why Grid exists, given that Flexbox already exists

Yesterday's Flexbox is fundamentally **one-dimensional** — at any given moment, you're arranging items along a single main axis (a row, or a column), even though items can visually wrap onto multiple lines. **CSS Grid is two-dimensional** — you define rows and columns **together**, as one coordinated structure, and can place items into specific cells of that structure precisely. This makes Grid the better tool for genuine page-level or section-level layouts (an overall page structure with a header, sidebar, main content, and footer, for instance), while Flexbox tends to shine at arranging a row or column of related items (a navbar, a button group, a list of cards) — you'll get a clear, direct answer to "which one should I use" at the end of today's lesson.

The core mental model: defining a grid

```
.container {  
  display: grid;  
  grid-template-columns: 200px 200px 200px;    /* THREE columns, each exactly 200px wide */  
  grid-template-rows: 100px 100px;             /* TWO rows, each exactly 100px tall */  
}
```

The instant you set `display: grid`, the direct children of `.container` automatically become grid items and, by default, fill the grid's cells in order — left to right, then wrapping to the next row — very similar to how text fills a page.

The `fr` unit — Grid's most distinctive, useful tool

Rather than only specifying fixed pixel widths, Grid introduces a special unit, `fr` (short for "fraction"), representing a share of whatever space remains after any fixed-size columns/rows are accounted for:

```
.container {  
  display: grid;  
  grid-template-columns: 1fr 1fr 1fr;    /* THREE equal-width columns, each taking an equal 1/3 share of space */  
}  
  
.container {
```

```

    display: grid;
    grid-template-columns: 200px 1fr;    /* a FIXED 200px sidebar, then the remaining space fills the rest */
}

```

This second example — one fixed-width column plus one `1fr` column — is an extremely common real layout: a sidebar with a fixed, predictable width, and a main content area that automatically fills whatever space remains, however wide or narrow the browser window happens to be. You can mix as many fixed and `fr` values as you like: `grid-template-columns: 150px 1fr 2fr;` gives a fixed 150px column, then splits the *remaining* space into three shares, giving one share to the second column and two shares (twice as much space) to the third.

`gap` — identical to Flexbox

```

.container {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  gap: 20px;    /* space between EVERY row and column gap, simultaneously */
}

```

You can also control row and column gaps independently with `row-gap` and `column-gap`, if you need them to differ.

Placing items precisely with grid lines

Grid automatically creates numbered **grid lines** at the edges of every row and column — for a 3-column grid, there are 4 vertical grid lines (numbered 1 through 4: before column 1, between 1 and 2, between 2 and 3, and after column 3). You can explicitly place any item to span specific lines, completely overriding the automatic left-to-right filling behavior:

```

.container {
  display: grid;
  grid-template-columns: repeat(3, 1fr);    /* shorthand for 1fr 1fr 1fr -- "repeat this pattern 3 times" */
  gap: 20px;
}
.featured-item {
  grid-column: 1 / 3;    /* span from grid line 1 to grid line 3 -- i.e., occupy the first TWO columns */
}

```

`repeat(3, 1fr)` is worth calling out on its own: it's shorthand meaning "repeat this pattern 3 times," and is far more common in real code than manually writing out `1fr 1fr 1fr` — you'll use `repeat(...)` constantly once you're building anything with more than 2-3 columns.

`grid-template-areas` — naming regions of your layout directly

This is, genuinely, one of CSS's most beginner-friendly, readable features, because it lets your CSS visually *look like* the layout it produces:

```

.page {
  display: grid;
  grid-template-columns: 200px 1fr;
}

```

```

grid-template-rows: auto 1fr auto;
grid-template-areas:
  "header header"
  "sidebar main"
  "footer footer";
gap: 15px;
min-height: 100vh;
}
.header { grid-area: header; }
.sidebar { grid-area: sidebar; }
.main { grid-area: main; }
.footer { grid-area: footer; }

```

Each quoted line in `grid-template-areas` represents one entire row of the grid, and each word represents one cell in that row — repeating the same name across multiple cells (like `header` appearing twice in the first row) makes that one named area span all of those cells. Then, each individual child element just declares `grid-area: header;` (or whichever name applies) to be placed into that named region — no manual line-number counting required at all, and the CSS itself visually resembles a rough sketch of the actual page layout, which makes it easy to read back later and immediately understand the overall structure.

Grid vs. Flexbox — the actual decision, answered clearly

This is a genuinely common, entirely reasonable beginner question, and it deserves a direct, confident answer rather than "it depends" hand-waving:

- **Reach for Grid** when you're laying out a genuine two-dimensional structure — rows AND columns need to be coordinated together, such as an overall page layout (header/sidebar/main/footer), a photo gallery, or a dashboard of cards arranged in a grid.
- **Reach for Flexbox** when you're arranging a single row OR a single column of related items, where the *other* dimension doesn't really need independent control — a navbar, a button group, a list of tags, centering one thing inside another.
- **They are frequently used together, at different levels of the same page** — a very common, entirely normal real-world pattern is Grid for the page's overall structure, with Flexbox used *inside* one of those grid areas (say, inside the header) to arrange its own internal row of items. Neither tool "replaces" the other; they solve different shaped problems, and most real, modern websites use both.

Exercises

Open `starter.html` and `starter.css`, follow the `<!-- TODO -->` `/* TODO */` markers, and check your work against `CHECKLIST.md`.